

Table of Contents

Unit 4: Computational Structures	2
An 11th Class Computer Science Study Guide	2
Introduction	2
4.1 Primitive Computational Structures	2
4.1.1 Lists	3
4.1.1.1 List Creation	3
4.1.1.2 List Properties	4
4.1.1.3 List Operations	5
4.1.1.4 Applications of Lists	6
4.1.2 Stacks	7
4.1.2.1 Stack Operations	7
4.1.3 Queues	9
4.1.3.1 Queue Operations	9
4.1.4 Trees	11
4.1.4.1 Properties of Trees	11
4.1.4.2 Applications of Trees	13
4.1.5 Introduction to Graphs	14
4.1.5.1 Characteristics of Graphs	15
4.1.5.2 Properties of Graphs	15
4.1.5.3 Types of Graphs	16
Chapter Summary — The Big Picture	18
End-of-Chapter Exercises	19
Multiple Choice Questions	19
Short Questions	19
Long Questions	20

Unit 4: Computational Structures

An 11th Class Computer Science Study Guide

Introduction

Welcome, computational architect.

Yes — that's the right title for you now. Up until this point, you've been writing instructions: variables, loops, decisions, functions. But there's a question we haven't tackled yet, and it's one of the biggest questions in all of computer science:

Where do you actually put your data, and how do you organize it, so that your program can use it quickly and correctly?

Imagine two students studying for the same exam. One shoves every note, handout, and page into a single giant backpack with no order. The other organizes their notes into labeled folders, sorted by chapter, with an index at the front. Both students have the *same information*. But one of them will find what they need in two seconds, and the other will be digging through their bag for ten minutes, sweating, while the exam clock ticks.

That's the entire idea behind this unit. A **computational structure** (also called a **data structure**) is simply an organized way of storing data so it can be accessed and used efficiently. In this unit, we'll explore five foundational structures: **lists**, **stacks**, **queues**, **trees**, and **graphs**. Each one is a different way of organizing the same raw material — data — for a different purpose.

By the end of this chapter, you won't just know *what* these structures are. You'll know *when* to reach for each one, the same way a carpenter knows exactly which tool to pick up for which job. Let's begin building your toolbox.

4.1 Primitive Computational Structures

There are several commonly used computational structures that form the foundation of almost every computer program ever written. Let's meet them one at a time.

4.1.1 Lists

4.1.1.1 List Creation

The Hook (Story Mode):

Open your favorite music app right now — Spotify, YouTube Music, whatever you use. Look at your playlist. Song 1, Song 2, Song 3... each one sits in a specific position. You can drag a song to reorder it, delete a song you're tired of, or add a brand-new one to the end. That playlist, with its ordered, changeable sequence of songs, is a perfect real-world **list**.

The Explanation:

A **list** is a data structure used to store multiple pieces of data in a specific sequence. Each individual piece of data is called an **element**, and every element sits at a particular position, called its **index**, inside the list. This positioning is what makes lists so easy to access and manage.

In Python, lists are created using square brackets `[]`, with each item separated by a comma:

```
# Create a list of items
items = ["Decorations", "Snacks", "Cold drinks", "Plates", "Balloons"]

# Print the list
print(items)
```

Explanation: `items` is the name of the list. The individual elements — `"Decorations"`, `"Snacks"`, `"Cold drinks"`, `"Plates"`, and `"Balloons"` — are each enclosed in quotes (since they're text) and separated by commas.

The Practical Walkthrough:

1. Open a Python file called `list_creation.py`.
2. Type the `items` list exactly as shown above.
3. Print the list, and then print a single element using its index, like `print(items[2])`.
4. Run the file and observe both outputs.

What just happened? You created your very first computational structure — an ordered container holding five separate pieces of data, each one individually retrievable by its position.

Interactive Stop-Point (Pause & Think):

If you added a sixth item, `"Music Speaker"`, to the end of your `items` list, what index number would it receive? What does this tell you about how Python numbers list positions?

Quick Recap:

A list stores multiple elements in a specific, ordered sequence, and each element can be individually accessed using its index position.

4.1.1.2 List Properties

The Hook (Story Mode):

Think again about that playlist. It's not a fixed, unchangeable object — it's *alive*. You can add ten more songs tonight. You can delete three you don't like anymore. And no matter what you do, the app never forgets the order you arranged things in. These three behaviors are exactly the three core properties of a list.

The Explanation:

A list has three defining properties:

1. **Dynamic Size** — A list can grow or shrink. You can add new items or remove existing ones without any problem, and the list automatically adjusts to fit the change. You never need to declare, in advance, exactly how many items it will hold.
2. **Index-Based Access** — Every item has a position, called an index. The first item is at index `0`, the second at index `1`, and so on. You can use these indexes to directly retrieve any specific item.
3. **Ordered Collection** — The order in which you add items is preserved. If you add an item first, it stays in that position unless you deliberately change it.

Here's a simple memory map of our `items` list:

Index:	0	1	2	3	4
Value:	"Decorations"	"Snacks"	"Cold drinks"	"Plates"	"Balloons"

The Practical Walkthrough:

1. In your `list_creation.py` file, add a new item to the end of the `items` list using `items.append("Music Speaker")`.
2. Print the list again — notice the new size.
3. Print `items[0]` and `items[4]` to confirm the original ordering was preserved.

What just happened? You directly observed all three properties in action: the list's size grew dynamically, each element remained retrievable by its fixed index, and the original order of the first five items never changed.

Interactive Stop-Point (Pause & Think):

If a list's size can change freely, why do we still bother calling it an "ordered" collection? Isn't "ordered" the opposite of "flexible"? Explain the difference between order and size.

Quick Recap:

Lists are dynamic (they can grow or shrink), index-based (every element has a retrievable position), and ordered (the sequence you build is preserved until you change it).

4.1.1.3 List Operations

The Hook (Story Mode):

Think about managing a real to-do list for a birthday party. You add new tasks as you think of them. You cross off tasks once they're done. And sometimes, you scan the whole list just to check: "Wait, did I already write down 'buy balloons'?" These three everyday actions map directly onto the three core list operations: **insertion**, **deletion**, and **searching**.

The Explanation:

1. Insertion

Adding a new item to a list is like adding a new task to your to-do list. You can insert an item at any specific position using the `insert()` function.

```
party_list = ["Buy drinks", "Buy decorations", "Buy snacks", "Buy cold drinks"]
party_list.insert(0, "Invite friends") # add "Invite friends" at the very start
print(party_list)
# Output: ['Invite friends', 'Buy drinks', 'Buy decorations', 'Buy snacks', 'Buy cold drinks']
```

2. Deletion

Removing an item from a list is like crossing off a completed task. There are two common ways:

- **Removing by Value** — use `remove()` to delete the *first occurrence* of a specific item.

```
python
```

```
party_list = ["Invite friends", "Buy decorations", "Buy snacks", "Buy cold drinks"]
party_list.remove("Buy snacks") # Removes "Buy snacks" from the list
print(party_list)
# Output: ['Invite friends', 'Buy decorations', 'Buy cold drinks']
```

- **Removing by Index** — use `pop()` to remove the item sitting at a specific index.

```
python
```

```
party_list = ["Invite friends", "Buy decorations", "Buy cold drinks"]
party_list.pop(0) # Removes the item at index 0
print(party_list)
# Output: ['Buy decorations', 'Buy cold drinks']
```

3. Searching

Finding an item in a list is like scanning your to-do list for a specific task. Use the `in` keyword to check whether an item exists:

```
party_list = ["Invite friends", "Buy decorations", "Buy cold drinks"]
if "Buy cold drinks" in party_list:
    print("Buy cold drinks is on the list.")
else:
    print("Buy cold drinks is not on the list.")
# Output: Buy cold drinks is on the list.
```

The Practical Walkthrough — Full State Trace:

Let's trace every step of the insertion example, showing the list's state at each moment:

Step	Operation	List State
Start	—	<code>["Buy drinks", "Buy decorations", "Buy snacks", "Buy cold drinks"]</code>
1	<code>party_list.insert(0, "Invite friends")</code>	<code>["Invite friends", "Buy drinks", "Buy decorations", "Buy snacks", "Buy cold drinks"]</code>

1. Create `list_operations.py`.
2. Type the insertion example and run it, confirming the table above.
3. Add the deletion-by-value example directly below it, and predict the output before running.
4. Add the deletion-by-index example, and predict the output before running.
5. Add the searching example, changing the searched item to something *not* in the list, and observe the `else` branch execute.

What just happened? You practiced all three fundamental list operations — insertion, deletion (by both value and index), and searching — and confirmed each one's exact effect on the list's state.

Interactive Stop-Point (Grab a Partner):

Partner A writes a list of five grocery items. Partner B calls out three operations, one at a time — for example, "insert 'Milk' at index 2," "remove 'Bread' by value," "search for 'Eggs'" — and Partner A must correctly predict and then verify the list's new state after each one.

Quick Recap:

Insertion (`insert()`, `append()`), deletion (`remove()`, `pop()`), and searching (`in`) are the three fundamental operations that let you build, maintain, and query a list.

4.1.1.4 Applications of Lists

The Explanation:

Lists are one of the most widely used data structures in all of computing because of how flexible they are. Two major applications include:

- **Data Storage and Manipulation:** Lists are commonly used to store and manage collections of data — records, entries, or values — allowing easy insertion, deletion, and access to elements.
- **Stack and Queue Implementations:** Lists can serve as the underlying structure to implement stacks (LIFO) and queues (FIFO), two more specialized structures we're about to explore in the next two sections.

Interactive Stop-Point (Pause & Think):

Think of three apps on your phone that likely use a list "under the hood" to manage their data (hint: think about anything with a scrollable, ordered set of items — messages, photos, notifications). What do all three examples have in common?

Quick Recap:

Because of their flexibility, lists aren't just useful on their own — they're often the *building block* used to construct more specialized structures, like the stacks and queues we'll build next.

4.1.2 Stacks

4.1.2.1 Stack Operations

The Hook (Story Mode):

In 1946, Alan Turing was already experimenting with "reverting" storage registers — a way of storing values so the most recently saved one could be retrieved first. Just over a decade later, in 1957, German computer scientist **Friedrich L. Bauer** formally patented the **stack** data structure, designing it specifically to help evaluate arithmetic expressions using a Last-In, First-Out approach. That single idea — the *last thing in is the first thing out* — is still running, quietly, inside every calculator, compiler, and web browser you use today.

Here's an example you already know intuitively: the cafeteria tray dispenser. Trays get stacked one on top of another. When someone needs a tray, they take the one sitting on *top* — the very last one that was placed there. Nobody digs to the bottom of the pile.

The Explanation:

A **stack** is a data structure where items can only be added or removed from one end, called the **top**. Both insertion and deletion happen exclusively at this top end. A stack follows the **LIFO (Last-In, First-Out)** principle: the most recently added element is always the first one removed.

There are two essential operations:

- **Push:** Adding an item to the top of the stack.

- **Pop:** Removing the item from the top of the stack.

Two additional, very common supporting operations:

- **Peek (or Top):** Looking at the item currently on top, *without* removing it.
- **IsEmpty:** Checking whether the stack currently has zero items in it.

```
# Create an empty stack of books
stack_of_books = [] # Empty stack
print("Initial stack:", stack_of_books)

# Add books to the stack (push operation)
print("\nAdding books to the stack (push operation):")
stack_of_books.append('Book A')
print("Stack after pushing 'Book A':", stack_of_books)

stack_of_books.append('Book B')
print("Stack after pushing 'Book B':", stack_of_books)

# Remove the top book from the stack (pop operation)
print("\nDeletion of top book (pop operation):")
top_book = stack_of_books.pop()
print("Removed book:", top_book)
print("Stack after popping the top book:", stack_of_books)
```

Explanation: The code creates an empty stack to hold books. It adds "Book A" and "Book B", one at a time, to the top of the stack. Finally, it removes the top book, "Book B" — proving that the *last* book pushed onto the stack is the *first* one popped off.

Visualizing the "top" pointer as the stack changes:

Step 0: Empty stack	[]	top -> (nothing)
Step 1: Push "Book A"	["Book A"]	top -> "Book A"
Step 2: Push "Book B"	["Book A", "Book B"]	top -> "Book B"
Step 3: Pop()	["Book A"] (removed and returned "Book B")	top -> "Book A"

The Practical Walkthrough:

1. Create `stack_demo.py`.
2. Type the code above and run it, line by line, confirming each printed state against the table above.
3. Add a **Peek** operation manually by printing `stack_of_books[-1]` *without* calling `.pop()` — confirm the top item is visible but the stack size is unchanged.
4. Add an **IsEmpty** check using `if len(stack_of_books) == 0:`.

What just happened? You traced a stack through push, pop, and peek operations, watching the "top" position shift with every single change — exactly the mental model every programmer uses when reasoning about stacks.

Interactive Stop-Point (Pause & Think):

You are designing an "Undo" feature for a text editor, like Word or Google Docs. Should you use a **Stack** or a **Queue** to store the user's past actions? Think carefully about which action gets reversed *first* when the user clicks "Undo," and explain your reasoning.

Quick Recap:

A stack is LIFO — Last-In, First-Out — like a stack of trays or books: whatever you push on top is exactly what comes off first when you pop.

4.1.3 Queues

4.1.3.1 Queue Operations

The Hook (Story Mode):

Back in the 1960s, early computer networks and printer systems faced a real, practical problem: a fast processor could generate data far quicker than a slow printer could physically print it. If the processor simply blasted data at the printer, information would be lost. The solution? A **queue** — a waiting line where data patiently lines up, in the exact order it arrived, until the slower device is ready to handle it. This same principle protects your data every single time you send multiple print jobs, or send multiple messages faster than a server can process them.

You already know this pattern from real life: standing in line at a bank or a ticket counter. The first person to join the line is the first person served. Nobody who joins later gets served before someone who's already been waiting.

The Explanation:

A **queue** works exactly like that line. It follows the **FIFO (First-In, First-Out)** principle: the first item added is the first item removed. You add items at the **back** (or **rear**) of the queue, and remove items from the **front**.

The two primary operations:

- **Enqueue (Add an Item):** Like adding a person to the end of the line.
- **Dequeue (Remove an Item):** Like serving the person at the front of the line.

Additional common operations include checking if the queue is empty, peeking at the front item without removing it, and checking the queue's current size.

```

# Built-in module to implement queues in Python
from queue import Queue

# Create a new queue
q = Queue()

# Add people to the queue (Enqueue)
q.put("Ahmed")      # Adds Ahmed to the end of the queue
q.put("Fatima")     # Adds Fatima to the end of the queue

# View the person at the front of the queue (Peek)
front_person = q.queue[0]  # Looks at the person at the front without removing
                           # them
print(front_person)

# Remove a person from the front of the queue (Dequeue)
removed_person = q.get()    # Removes and returns the person at the front
print(removed_person)

# Add another person to the queue (Enqueue)
q.put("Sara")          # Adds Sara to the end of the queue

# View the updated queue
updated_queue = list(q.queue)
print(updated_queue)

```

Explanation: The code manages a line of people using a queue. It adds people to the back of the line, checks who is currently at the front without removing them, and then serves (removes) that front person. Finally, it adds a new person to the back and shows the updated line.

Visualizing `front` and `rear` as the queue changes:

Step 0: Empty queue	<code>[]</code>	front -> none rear -> none
Step 1: Enqueue "Ahmed"	<code>["Ahmed"]</code>	front -> Ahmed rear ->
Ahmed		
Step 2: Enqueue "Fatima"	<code>["Ahmed", "Fatima"]</code>	front -> Ahmed rear ->
Fatima		
Step 3: Peek (front)	<code>["Ahmed", "Fatima"]</code>	(no change; just viewed
"Ahmed")		"Ahmed")
Step 4: Dequeue()	<code>["Fatima"]</code>	front -> Fatima rear ->
Fatima		
		(removed and returned "Ahmed")
Step 5: Enqueue "Sara"	<code>["Fatima", "Sara"]</code>	front -> Fatima rear ->
Sara		

The Practical Walkthrough:

1. Create `queue_demo.py`.
2. Type the code above exactly and run it.
3. Compare each printed line against the state table above.
4. Add a check using `q.empty()` to confirm whether the queue is currently empty.

What just happened? You traced a real queue through enqueue, peek, and dequeue operations, and watched the `front` shift forward while the `rear` extended — the exact opposite behavior pattern from a stack.

Interactive Stop-Point (Grab a Partner):

Partner A acts as a printer buffer (a queue). Partner B sends five print commands, one at a time, each with a different "document name." Partner A must physically write down (or say out loud) the queue's state after each new command arrives, and correctly announce which document gets printed first when "processing" begins.

Quick Recap:

A queue is FIFO — First-In, First-Out — like a line at a ticket counter: whoever joins first is served first, with new arrivals always waiting at the back.

4.1.4 Trees

4.1.4.1 Properties of Trees

The Hook (Story Mode):

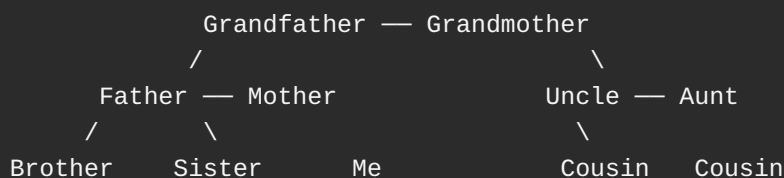
In 1969, engineers building the UNIX operating system faced a huge organizational challenge: how do you organize potentially millions of files on a single computer, in a way that's easy to navigate? Their answer became one of the most influential ideas in computing history — a **hierarchical tree structure**, where every single file and folder ultimately branches out from one single starting point: the root directory, . Every time you open a folder on your computer today, you're navigating a tree that traces its design straight back to that 1969 decision.

You've also lived this structure your whole life, in a completely different context: your **family tree**. There's a starting generation at the top, and it branches downward into children, grandchildren, and so on.

The Explanation:

A **tree** data structure organizes information so that it spreads out from a single main point called the **root node**. Each individual piece of information is called a **node**, and nodes connect to other nodes, forming a branching structure — very different from a list, where items sit one after another in a straight line.

Example: In a family tree, the oldest ancestors form the root node — the starting point of the entire hierarchy. Each individual may have descendants, forming further levels beneath them. This kind of complex, branching, parent-child relationship simply cannot be represented well in a flat, linear list — which is exactly why the tree structure exists.



Key properties of a tree:

1. **Root Node** — The very first, topmost node in a tree, similar to the main folder on a computer that contains every other folder and file within it.
2. **Edges and Nodes** — Nodes are the individual elements; the connecting lines between them are called **edges**. A node with no children is called a **leaf**, similar to a file that contains no other files inside it.
3. **Height** — The length of the longest path from the root node down to the farthest leaf. It tells you how "deep" or "tall" the tree is.
4. **Depth** — The distance (number of edges) from the root down to one *specific* node. (Height measures the tree's overall extent; depth measures one particular node's position within it — a distinction that trips up even university engineering students, so don't worry if it takes a few examples to click.)
5. **Subtrees** — Any node in a tree, along with all of its descendants, forms a smaller tree of its own, called a **subtree**. For example, "Father," "Brother," and "Sister" together form a subtree of the larger family tree above.
6. **Balanced Trees** — A tree is considered balanced if the branches on the left and right sides are nearly the same height. Balanced trees tend to be far more efficient to search than lopsided ones.

The Practical Walkthrough:

1. Look carefully at the family tree ASCII diagram above.
2. Identify the root node. (It should be "Grandfather — Grandmother," the topmost generation.)
3. Identify every leaf node — nodes with no children beneath them. (Brother, Sister, Me, and both Cousins should qualify.)
4. Count the height of the tree — the longest path from the root down to any leaf. (It should be 2 edges: Grandfather → Father → Me, for example.)
5. Identify the depth of the node "Me" specifically. (It should also be 2, since "Me" sits two edges below the root.)

What just happened? You practiced identifying every core structural property of a tree using a diagram you already intuitively understand — your own family structure.

Interactive Stop-Point (Pause & Think):

Look at the height calculation you just did. Now imagine a *different* family tree where one branch has five generations of descendants, but another branch has only one. Would this tree be "balanced"? Why does a balanced tree matter for how quickly you can search through it?

Quick Recap:

A tree branches outward from a single root node into connected nodes and edges; height measures the tree's overall depth, while depth measures one specific node's distance from the root.

4.1.4.2 Applications of Trees

The Hook (Story Mode):

Think about backing up your entire computer, folder by folder. If you accidentally tried to back up a folder's *contents* before confirming the folder itself exists in the backup, you'd create chaos. Trees solve this ordering problem elegantly, through structured traversal.

The Explanation:

Trees show up constantly in real-world computing:

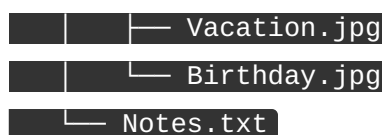
1. **File Systems (Backup):** *Pre-order traversal* — visiting the root first, then recursively visiting each branch — is useful for backing up file systems. By visiting the root first and then recursively backing up each directory, it ensures directories are backed up *before* their contents.
2. **File System Deletion:** *Post-order traversal* ensures files and directories are deleted in the correct order, by first deleting all subdirectories and files *before* deleting the parent directory — the reverse order from backup, and for good reason: you can't delete a folder that Windows or your OS still thinks contains files.
3. **Hierarchical Data Representation:** Trees represent data with a clear hierarchical relationship, such as organizational charts and family trees.
4. **Decision Making:** Structures called *decision trees* are used in algorithms to make decisions based on a chain of conditions and outcomes — a foundational idea you'll encounter again if you study artificial intelligence.

The Practical Walkthrough:

1. Imagine a simple folder structure:

Documents/

└── Photos/



2. Trace a **pre-order backup**: visit `Documents/` first, then `Photos/`, then `Vacation.jpg`, then `Birthday.jpg`, then finally `Notes.txt`.
3. Trace a **post-order deletion**: delete `Vacation.jpg` first, then `Birthday.jpg`, then `Photos/` (now empty), then `Notes.txt`, and only then `Documents/` itself.

What just happened? You practiced two opposite traversal orders and saw, very concretely, why the order of visiting nodes matters enormously depending on the real-world task (creating vs. destroying data).

Interactive Stop-Point (Pause & Think):

Why is a file folder system represented as a tree rather than a flat list? What specific problems would occur if a folder were somehow allowed to contain its own parent folder?

Quick Recap:

Trees aren't just theoretical — they power real backup systems, safe file deletion, organizational charts, and decision-making algorithms, all thanks to their strict, ordered, branching structure.

4.1.5 Introduction to Graphs

The Hook (Story Mode):

In 1736, mathematician **Leonhard Euler** was presented with a strange puzzle from the city of Königsberg: the city had seven bridges connecting different landmasses, and locals wondered whether it was possible to take a walk that crossed every single bridge exactly once, and return to your starting point. Euler realized something brilliant — the *exact shapes* of the landmasses didn't matter at all. All that mattered was which landmasses were *connected* to which others, and by how many bridges. He simplified each landmass into a single dot (what we now call a **vertex**), and each bridge into a line connecting two dots (what we now call an **edge**). In doing so, Euler proved the walk was impossible — and, almost as a side effect, he invented an entire new branch of mathematics: **graph theory**.

Every time you open Google Maps, or scroll through your list of followers on social media, you are interacting directly with Euler's 1736 idea.

The Explanation:

A **graph** is a data structure made up of a set of **vertices** (also called **nodes**) connected by **edges**. Graphs represent networks of connections, where each edge represents a relationship between two vertices. Vertices can represent anything — cities, people, web pages, or even abstract ideas — and edges represent the relationships or pathways between them.

Imagine mapping out all the cities in Pakistan and the roads connecting them. Each city is a vertex; each road between two cities is an edge. Unlike a tree, a graph doesn't have a single "root" and doesn't follow a strict hierarchy. In a graph, any two vertices can potentially be connected, forming a complex web of relationships rather than a clean branching structure.

Example: In a social network, each person can be connected to many others, forming a graph. There's no single starting point, and people (vertices) can have multiple connections (edges) that don't follow any strict parent-child relationship, unlike a tree.

Difference from a Tree:

Feature	Tree	Graph
Root node	Exactly one, always	Not required — may have none
Hierarchy	Strict, top-down	None required
Paths between two nodes	Exactly one (no cycles/loops)	Can be multiple, and cycles (loops) are allowed
Typical real-world use	Family trees, org charts, file systems	Social networks, web links, transport/road systems

4.1.5.1 Characteristics of Graphs

Graphs are defined at their core by their two basic building blocks:

- **Vertices (Nodes):** The individual points in the graph — the "things" being connected (cities, people, pages).
- **Edges:** The connections between two vertices — the "relationships" or "links" between those things.

4.1.5.2 Properties of Graphs

Beyond their basic building blocks, graphs have several structural properties worth knowing:

- **Degree:** The number of edges connected to a specific vertex. For example, if a city is connected to three other cities by roads, that city's vertex has a degree of **3**.
- **Path:** A sequence of edges that lets you travel from one vertex to another.
- **Cycle:** A path that starts and ends at the *same* vertex, forming a loop. (Recall: trees never allow cycles, but graphs do.)
- **Connectedness:** A graph is considered "connected" if there is *some* path between every pair of vertices — no vertex is completely isolated from the rest.
- **Weight:** In some graphs, edges carry a weight representing a value like distance or cost. For example, if a road between two cities is 50 kilometers long, that edge's weight might be **50**.

- **Direction:** Edges can be directed (one-way) or undirected (two-way). A directed edge from city A to city B does not automatically mean there's a return edge from B to A.

4.1.5.3 Types of Graphs

Graphs are classified based on their structure and properties. The three main types:

1. Directed Graphs

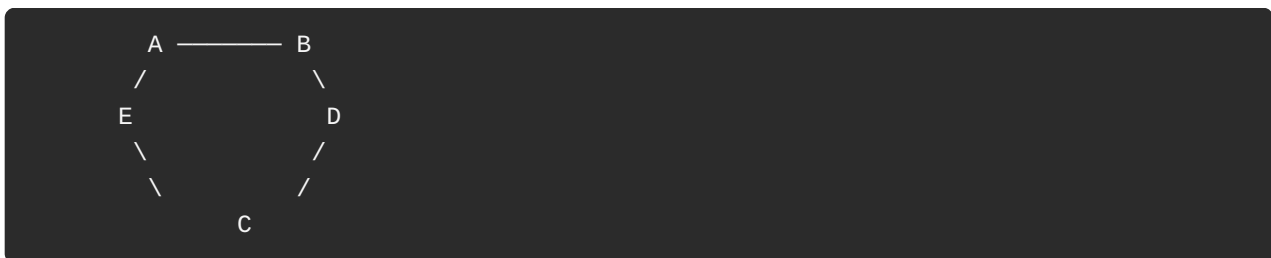
In a directed graph, edges have a specific direction — they travel from one vertex to another in only one permitted way.



Example: If you want to travel from city A to city B, you can only go in the direction the road allows. If there's no one-way street running from A to B, you simply cannot travel directly that way, even if a road exists in the reverse direction.

2. Undirected Graphs

In an undirected graph, edges have no direction — a connection between two vertices allows travel in both directions equally.



Example: If Person A is friends with Person B on a social network, then Person B is automatically friends with Person A too. There's no restriction on direction — you can move freely between connected friends.

3. Weighted Graphs

In a weighted graph, each edge carries a weight or cost representing something meaningful, like distance, time, or price.

Example: Imagine a city map where every road has a different travel distance or time attached. If you want to travel between two landmarks, the map uses these weights to calculate your shortest or quickest possible route — exactly what a navigation app does behind the scenes every single time you request directions.

The Practical Walkthrough — Representing a Small Graph:

Let's represent a tiny social graph of four friends — Ali, Bilal, Sara, and Zara — where Ali is friends with Bilal and Sara, and Bilal is friends with Zara.

As an Adjacency List (each vertex lists its direct connections):

```
Ali    -> [Bilal, Sara]
Bilal  -> [Ali, Zara]
Sara   -> [Ali]
Zara   -> [Bilal]
```

As an Adjacency Matrix (a grid showing 1 if connected, 0 if not):

	Ali	Bilal	Sara	Zara
Ali	0	1	1	0
Bilal	1	0	0	1
Sara	1	0	0	0
Zara	0	1	0	0

1. Build both representations on paper for the friendship graph described above.
2. Confirm that both representations agree with each other — anywhere the adjacency list shows a connection, the matrix should show a **1** in the matching cell.

What just happened? You represented the exact same graph in two different, commonly used formats — proving that a graph's "shape" can be captured in more than one valid way, depending on what your program needs to do with it.

Interactive Stop-Point (Pause & Think):

For a flight navigation app showing direct flights between cities, should the graph be **directed or undirected**? Should it be **weighted or unweighted**? Justify your choice using a real detail about how flights actually work (hint: think about one-way flights and ticket prices).

Quick Recap:

A graph is a flexible network of vertices connected by edges — which may be directed or undirected, and weighted or unweighted — making it the ideal structure for modeling real-world networks like roads, social connections, and flight routes, where a strict tree hierarchy simply wouldn't fit.

Chapter Summary — The Big Picture

Let's step back and look at the whole toolbox you just built:

- **Lists** give you a flexible, ordered, index-based way to store and manage a general collection of data — and they can even serve as the raw material to build other structures.
- **Stacks** enforce LIFO (Last-In, First-Out) discipline — perfect for anything involving "undo" actions, reversing steps, or evaluating expressions, using just two core moves: push and pop.
- **Queues** enforce FIFO (First-In, First-Out) discipline — perfect for anything involving fair waiting lines, task scheduling, or buffering, using enqueue and dequeue.
- **Trees** organize hierarchical, branching relationships from a single root — ideal for file systems, organizational charts, and decision-making logic, with properties like height, depth, and balance shaping how efficiently they perform.
- **Graphs** model flexible, non-hierarchical networks of connections — directed or undirected, weighted or unweighted — ideal for social networks, maps, and any system where relationships don't follow a strict chain of command.

Here's the real skill you've built in this unit: it's not memorizing five definitions. It's developing the instinct to look at a real-world problem and ask, "Which structure actually fits the *shape* of this data?" That instinct — choosing the right tool before you even start coding — is one of the true marks of a skilled computational architect. Carry it forward.

End-of-Chapter Exercises

Multiple Choice Questions

1. The function used to add an item at the end of a list in Python:
a) `insert()` b) `append()` c) `remove()` d) `pop()`
2. The purpose of the `in` keyword used with a Python list:
a) Adds an item to the list b) Removes an item from the list c) Checks if an item exists in the list d) Returns the length of the list
3. An operation that removes an item from the top of the stack:
a) Push b) Pop c) Peek d) Add
4. The operation used to add an item to a queue:
a) Dequeue b) Peek c) Enqueue d) Remove
5. True statement about the height of a tree:
a) Number of edges from the root to the deepest node
b) Number of nodes from the root to the deepest node
c) Number of children of the root node
d) Always equal to the number of nodes in the tree
6. A scenario where a graph data structure is most suitable:
a) Managing a to-do list
b) Modeling a line of customers in a store
c) Representing connections in a social network
d) All of the above

Short Questions

1. Explain how the `insert()` function works in Python lists. Provide an example.
2. Explain the potential issues which could arise when two variables reference the same list in a program. Provide an example.
3. Define a stack and explain the Last-In, First-Out (LIFO) principle.
4. Differentiate between the Enqueue and Dequeue operations of a queue.
5. Name two basic operations performed on a stack.
6. What is the difference between `enqueue()` and `dequeue()`?

Long Questions

1. Discuss the dynamic size property of lists in Python. How does this property make lists more flexible?
2. Explain the operations on a stack with a real-life example and Python code.
3. Write a simple program to implement a queue (insertion and deletion).
4. Define a tree and explain its properties.
5. What is a graph? Explain the differences between directed and undirected graphs.